Day 143: Text-to-Image Models Conceptually: CLIP-Style Joint Embeddings, a Real Demo

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 21, Day 4 | Complete study guide

Watch: OpenAI's CLIP Explained and Implementation | Contrastive Learning (Karndeep Singh) | CLIP: Learning Transferable Visual Models From Natural Language Supervision (KnowledgeDB AI)

Script: `day143_clip_joint_embeddings.py` · every number in this guide is copied by program from `day143_run_output.txt`, the real output of that script.

Every generator so far chose its own digit, or took a digit label (Day 137). Text-to-image models take a sentence. Their first piece is usually a pair of encoders trained so that an image and a caption describing it land close together in one shared space: CLIP (Radford et al., 2021). Today's version is built from scratch on MNIST, with captions written by program from each digit's label and two measured attributes, thickness and lean. After 45 s of training, the pair names test digits from words alone with accuracy 0.9805, close to Day 138's trained classifier (0.9829), and finds thick, thin or leaning digits by description. Its limits are as instructive: word order, unknown words and the word 'not' change nothing at all.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

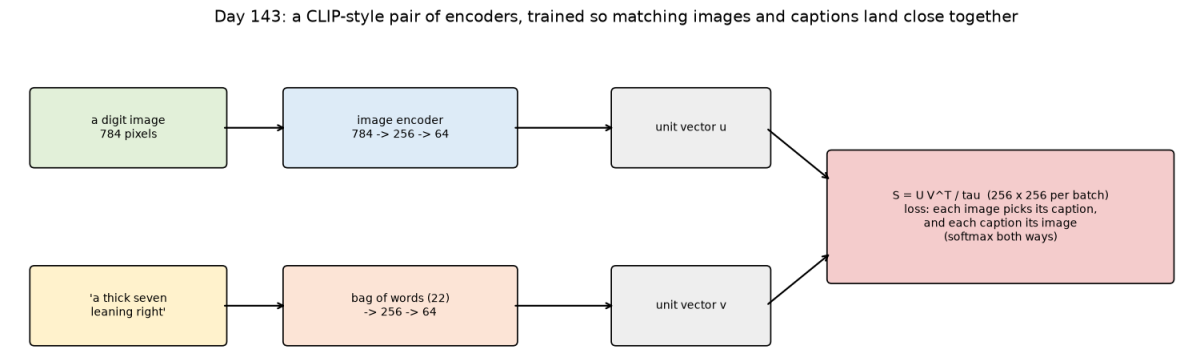
A **joint embedding** puts two kinds of thing, here images and captions, into one space of numbers, so that an image and a caption that describes it are close together. Two separate encoders do this: an **image encoder** turns 784 pixels into 64 numbers, a **text encoder** turns a caption into 64 numbers, and both outputs are scaled to length 1.

Contrastive training shows the pair a batch of matching image-caption pairs and asks each image to pick out its own caption from all the captions in the batch, and each caption its own image. Nothing tells the encoders what a 'seven' is; they only learn which images and which words go together.

REAL-WORLD SCENARIO

A photo library you can search by typing 'dog on a beach at sunset', though nobody ever tagged a photo: the search compares the typed words with every photo in one shared space. Text-to-image generators use the same kind of text encoder to tell the image model what to draw (Day 144).

SYSTEM DIAGRAM



Two encoders, one shared space: a batch's 256 images and 256 captions give a 256 × 256 table of similarities, and the loss asks the matching pairs (the diagonal) to win.

Today's part for the week's build

`day143_exercise.py` is today's parts (the measured attributes, the caption writer, the bag of words, the contrastive loss, the training loop, both encoders' embeddings and zero-shot naming), checked by hand: the gradient against finite differences, the loss of perfect and of useless embeddings, one caption's bag of words, the slant of made-up strokes, and one epoch of training on 10,000 digits.

ACTUAL OUTPUT

```
gradient check, image side: 0.12419889 against 0.12419889; text side: -0.02282094 against
    -0.02282094
8 perfectly matched pairs, tau 0.1: loss 0.000318; all embeddings equal: loss 2.0794 (log 8 =
    2.0794)
bag of words of 'seven thick seven zebra': seven 0.667, thick 0.333, total 1.000 ('zebra' is not
    in the vocabulary)
slant of a stroke leaning right: 0.335; leaning left: -0.335 (about 1/3 of a column per row,
    either way)
one epoch on 10,000 digits: loss 3.9667 (log 256 = 5.5452 for guessing); zero-shot accuracy on
    2,000 test digits 0.898
```

Data: Day 126's 70,000 MNIST digits, pixels in [0, 1] (training images 0–59,999; test images 60,000 on, taken in random order where a sample is used). The networks and Adam are Day 126's, imported unchanged. CLIP itself was trained on 400 million image-caption pairs from the web, with a Transformer text encoder; today's text encoder is a bag of words, which is part of what section 4 tests.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. By hand: the contrastive loss on three pairs

DEFINITION — start here, no prior knowledge assumed

Each image and caption becomes a unit vector. Their **similarity** is the dot product (the cosine of the angle between them), divided by a **temperature** τ that sharpens the softmax. For each image, a softmax over all captions in the batch gives the probability of picking each one; the loss is minus the log probability of the right one, averaged, and the same is done from each caption's side.

REAL-WORLD SCENARIO

A matching game at a party: each guest holds a photo, each card on the table holds a description, and every guest must find their card while every card is matched to its guest. Mistakes in either direction cost points.

MATHEMATICS

$$u_i = z_i / |z_i|, \quad v_j = w_j / |w_j|, \quad S_{ij} = u_i \cdot v_j / \tau$$

$$loss = -(1/2n)\, \Sigma_i \, [\, log\ softmax_j(S_{ij})_{j=i} + log\ softmax_i(S_{ij})_{i=j} \,]$$

$$dS = (P_{row} - I + P_{col} - I) / 2n; \quad dz = (du - u\,(u \cdot du)) / |z| \text{ (through the scaling to length 1)}$$

IMPLEMENTATION

```
def clip_loss(Zi, Zt, tau=TAU):
    ni = np.linalg.norm(Zi, axis=1, keepdims=True) + 1e-12; nt = np.linalg.norm(Zt, axis=1,
        keepdims=True) + 1e-12
    U, V = Zi / ni, Zt / nt; S = U @ V.T / tau; n = len(S); I = np.arange(n)
    P1 = np.exp(S - S.max(1, keepdims=True)); P1 /= P1.sum(1, keepdims=True)
    P2 = np.exp(S - S.max(0, keepdims=True)); P2 /= P2.sum(0, keepdims=True)
    loss = float(-(np.log(P1[I, I] + 1e-30).mean() + np.log(P2[I, I] + 1e-30).mean()) / 2)
    G1 = P1.copy(); G1[I, I] -= 1; G2 = P2.copy(); G2[I, I] -= 1; dS = (G1 + G2) / (2 * n)
    dU = dS @ V / tau; dV = dS.T @ U / tau
    return loss, (dU - U * (U * dU).sum(1, keepdims=True)) / ni, (dV - V * (V * dV).sum(1,
        keepdims=True)) / nt
```

ACTUAL OUTPUT

```
three images and three captions, unit vectors in 2-D; pair i is image i with caption i; temperature 1
similarities (rows images, columns captions):
    0.80  0.00  1.00
    0.60  1.00  0.00
    0.96  0.80  0.60
image 0 picks caption 0 with probability e^0.8 / (e^0.8 + e^0 + e^1) = 0.3744; caption 0 picks image 0 with 0.3342
loss: code 0.996814, by hand 0.996814 (the average of the six -log probabilities)
gradient against finite differences, all 48 entries of two random 6 x 4 batches, temperature 0.1: largest gap 9.93e-10
Actual output from this lesson's run.
```

What it means

Image 0 is closer to caption 2 (similarity 1.00) than to its own caption (0.80), so it picks its own with probability only 0.3744; the code's loss and the hand computation agree (0.996814). The gradient, which includes the step through the scaling to unit length, matches finite differences on all 48 entries to 9.93e-10.

WHY CHOOSE THIS?

One loss trains both encoders at once, needs no labels beyond 'these belong together', and every other pair in the batch serves as a free wrong answer.

WHY NOT THIS? (TRADEOFFS)

Each image is compared only with the captions in its batch; captions that describe the image equally well (two 'a seven's) count as wrong answers.

2. Captions from measurements

DEFINITION — start here, no prior knowledge assumed

MNIST has labels but no captions, so they are written by program. Two attributes are measured from the pixels: **thickness** (the average pixel: more ink means thicker strokes) and **lean** (how far the ink's column moves per row going up). Each is cut into thirds within its own digit, so 'thick' means thick for a seven, not compared with an eight. Each caption names the digit and, at random, its thickness, its lean and a filler word.

REAL-WORLD SCENARIO

Writing alt text for thousands of product photos from a spreadsheet: a template fills in the colour and size the database already holds, so every photo has a description, but only one written in a few patterns.

MATHEMATICS

$$thickness = mean\ pixel; \quad lean = -\Sigma\, p\,(r - r_0)(c - c_0) \,/\, \Sigma\, p\,(r - r_0)^2$$

p = pixel value, r, c = its row and column, r_0, c_0 = the ink's centre; positive lean: the top is further right

IMPLEMENTATION

```
def caption(d, th, ln, rng, hold=None):
    w = [NUM[d]]
    if th != 0 and rng.rand() < 0.6 and (d, th) != hold:
        w.append("thick" if th > 0 else "thin")
    if ln != 0 and rng.rand() < 0.6:
        w += ["leaning", "right" if ln > 0 else "left"]
    elif ln == 0 and rng.rand() < 0.3:
        w.append("upright")
    return w + [["a"], ["the", "digit"], ["handwritten"], ["written"], []][rng.randint(5)]
```

ACTUAL OUTPUT

```
thickness: the average pixel; lean: how far the ink's column moves per row (positive: leans right); each cut into
thirds within its own digit, from the 60,000 training digits, so 'thick' means thick for that digit
 digit  ink: thin below  thick above  lean: left below  right above
  zero           0.1539       0.1890            0.1194       0.3122
   one           0.0646       0.0829            0.0842       0.3561
   two           0.1310       0.1635           -0.0214       0.2078
 three           0.1230       0.1560           -0.0511       0.1589
  four           0.1051       0.1328           -0.0811       0.1905
  five           0.1104       0.1416            0.0632       0.3516
   six           0.1185       0.1506           -0.0688       0.1213
 seven           0.0988       0.1256           -0.0803       0.1444
 eight           0.1311       0.1636            0.0662       0.2881
  nine           0.1062       0.1325           -0.1079       0.1099
vocabulary: 22 words; six random captions for training digit 0 (five, thickness 0, lean 0):
   five the digit | five a | five written | five the digit | five upright a | five a
share of training digits that are thick, by the measure: 0.333; leaning right 0.333
Actual output from this lesson's run.
```

What it means

The cut points differ by digit, as intended: a one counts as thick above 0.0829, a zero only above 0.1890. A third of training digits are thick (0.333) and a third lean right, by construction. The six captions for one training five show how little each says, and how repetitive they are: the same five can be called 'five a' twice.

WHY CHOOSE THIS?

Thousands of captions for free, with attributes that can be checked against the pixels afterwards.

WHY NOT THIS? (TRADEOFFS)

Twenty-two words in a few patterns: nothing like the variety of real captions, so this tests the method, not language.

3. Training the two encoders

DEFINITION — start here, no prior knowledge assumed

Both encoders are Day 126's dense networks with one hidden layer of 256 ReLUs and 64 outputs. The text encoder's input is a **bag of words**: how often each of the 22 words appears in the caption, divided by the caption's length. Each batch is 256 training digits, each with a fresh random caption. After each epoch, **zero-shot** accuracy is measured: each of 2,000 test digits is named by whichever of the ten prompts 'the digit zero' ... 'the digit nine' is closest to it.

REAL-WORLD SCENARIO

Learning a language from a picture book with no translations: after enough pages, the word under every picture of a dog is the same, and the link forms without anyone defining 'dog'.

SPECIFICATION

Image encoder	784 → 256 (ReLU) → 64
Text encoder	22-word bag of words → 256 (ReLU) → 64
Training	Adam lr 0.001, batches of 256, temperature 0.1 (the only value tried), 10 epochs over 60,000 digits
Zero-shot	no classifier: an image takes the digit whose prompt embedding is most similar

IMPLEMENTATION

```
def train_clip(X, y, thick, lean, epochs=EPOCHS, hold=None, seed=0, batch=256, lr=1e-3,
    every=None):
    img = d126.init_net([784, 256, 64], seed); txt = d126.init_net([len(VOCAB), 256, 64], seed +
        1)
    oi, ot = d126.Adam(img, lr), d126.Adam(txt, lr); rng = np.random.RandomState(seed); log = []
    for ep in range(1, epochs + 1):
        o = rng.permutation(len(X)); L = []
        for s in range(0, len(X) - batch + 1, batch):
            b = o[s:s + batch]; caps = [caption(y[i], thick[i], lean[i], rng, hold) for i in b]
            Ai = d126.forward(img, X[b], ACTS); At = d126.forward(txt, bow(caps), ACTS)
            loss, gi, gt = clip_loss(Ai[-1].astype(np.float64), At[-1].astype(np.float64))
            oi.step(img, d126.backward(img, Ai, ACTS, gi.astype(np.float32))[0])
            ot.step(txt, d126.backward(txt, At, ACTS, gt.astype(np.float32))[0]); L.append(loss)
        log.append((ep, float(np.mean(L)), every(img, txt) if every else None))
    return img, txt, log
```

ACTUAL OUTPUT

```
image encoder 784 -> 256 -> 64; text encoder 22 (bag of words) -> 256 -> 64; both Day 126's networks and Adam (lr 0.001);
batches of 256 digits with a fresh random caption each; temperature 0.1; 10 epochs over the 60,000 training digits;
zero-shot accuracy after each epoch: 2,000 test digits named by the prompt 'the digit <word>'; a loss of log 256 = 5.545
    would mean no better than chance
 epoch     loss  zero-shot accuracy
     1   3.1678              0.9615
     2   2.8015              0.9705
     3   2.7381              0.9730
     4   2.6952              0.9785
     5   2.6705              0.9780
     6   2.6447              0.9785
     7   2.6276              0.9775
     8   2.6184              0.9800
     9   2.6071              0.9825
    10   2.5990              0.9805
(45 s, including the checks)
image -> caption among the same 2,000 test digits' captions: the exact caption ranked first 0.0325 (chance 0.0005); a
    caption of the right digit first 0.9790
Actual output from this lesson's run.
```

What it means

After a single epoch zero-shot accuracy was already 0.9615; after ten, 0.9805. No classifier was trained: the digit words were learned only as captions. The loss stayed far above zero (2.5990) and cannot reach it: in a batch of 256 there are about 25 of each digit, many with identical captions, so the right caption is often indistinguishable from several wrong ones.

The same limit shows when each image looks for its exact caption among 2,000 test captions: it ranks first only 0.0325 of the time, 65 times chance (0.0005) but low, while a caption of the right digit ranks first 0.9790 of the time. The encoders learned digit and attributes; the random filler words carry nothing to find.

WHY CHOOSE THIS?

A classifier for any set of words in the vocabulary, made by writing prompts rather than by training.

WHY NOT THIS? (TRADEOFFS)

It names only what the captions taught: a digit word that never appeared in a caption has no meaning to the text encoder.

4. Zero-shot classification, and what the prompt changes

DEFINITION — start here, no prior knowledge assumed

The same 2,000 test digits are named with nine different prompt templates, from '{}' (the bare digit word) to 'a photo of the number {}', plus an ensemble that averages five plain prompts. Two more checks compare prompts directly: the same words in a different order, and a prompt with unknown words added.

REAL-WORLD SCENARIO

Asking a shop assistant who speaks only a little of your language: 'a seven, please' and 'seven' get the same item, and 'not a seven, please' gets it too.

ACTUAL OUTPUT

```
2,000 test digits; each prompt is filled with the ten digit words and each image takes the most similar one.
For comparison, Day 138's trained classifier scored 0.9829 on the full test set.
                             prompt  accuracy   words the encoder does not know
                               '{}'    0.9800   -
                             'a {}'    0.9805   -
                       'the digit {}'    0.9805   -
                    'handwritten {}'    0.9795   -
          'a photo of the number {}'    0.9790   photo of number
                        'a thick {}'    0.9785   -
            'a thin {} leaning left'    0.9765   -
              '{} upright and thick'    0.9745   -
                         'not a {}'    0.9805   not
         ensemble of 5 plain prompts    0.9795
word order: 'a thin seven leaning left' against 'left seven thin leaning a': cosine 1.000000
unknown words: 'a seven' against 'a seven purple elephant': cosine 1.000000
Actual output from this lesson's run.
```

What it means

The prompt barely matters: every template scored between 0.9745 and 0.9805, a spread of 12 images in 2,000. Unknown words ('photo', 'of', 'number') are dropped by the bag of words, so they cost nothing. Adding attributes ('{} upright and thick', 0.9745) cost a little: the prompt then points partly towards thick digits of every class. Averaging five prompts did not help (0.9795).

Two results are exact by construction, and show what a bag of words cannot do: reordering the words gives cosine 1.000000 (the same embedding), and so does adding unknown words. 'not a {}' scored 0.9805, as well as 'a {}': 'not' is not in the vocabulary, and even if it were, a bag of words could not attach it to the digit. Real CLIP uses a Transformer that does see order, and prompt wording changes its accuracy by several points (Radford et al., 2021).

WHY CHOOSE THIS?

Changing the classifier is as cheap as rewording a sentence.

WHY NOT THIS? (TRADEOFFS)

The text encoder's limits become the classifier's: here no order, no negation, and nothing outside 22 words.

5. Finding digits by description

DEFINITION — start here, no prior knowledge assumed

The other direction: a caption is embedded, and the 10,000 test digits are ranked by similarity to it. For each query the top 50 are checked: are they the right digit, and does their measured attribute match the query? Each attribute covers a third of each digit, so a share of 0.33 is chance.

REAL-WORLD SCENARIO

Typing 'red running shoes' into a shop's search box and getting red running shoes, though no product was ever tagged 'red'.

IMPLEMENTATION

```
def retrieve(img, txt, X, words, pool, k=50):
    return pool[np.argsort(-(embed_images(img, X[pool]) @ embed_texts(txt,
        [words.split()])[0]))[:k]]
```

ACTUAL OUTPUT

```
top 50 for each query; 'right digit': share of them with that label; then the share whose measured attribute matches
the query (each attribute covers about a third of each digit, so 0.33 is what chance gives)
                    query  right digit  attribute matches  mean ink  mean lean
                    three         1.00                  -    0.1488     0.0672
            a thick three         1.00               1.00    0.2181     0.0673
             a thin three         1.00               1.00    0.0842     0.0095
        four leaning right         1.00               1.00    0.1177     0.4324
         four leaning left         1.00               1.00    0.1140    -0.3080
               a thick one         1.00               1.00    0.1022     0.2182
     a thin eight upright          1.00               1.00    0.1058     0.1743
all test threes: mean ink 0.1433, mean lean 0.0362
Actual output from this lesson's run.
```

What it means

Every query found the right digit in all 50 results, and every attribute query matched its attribute in all 50 (1.00 against 0.33 by chance). The measurements confirm it: 'a thick three' found threes with mean ink 0.2181 and 'a thin three' 0.0842, against 0.1433 for all test threes; 'four leaning right' and 'four leaning left' found lean 0.4324 and -0.3080.

A first draft of this check reported 0.00 for 'a thin eight upright': it tested for the word 'right' by looking inside the query's text, and found it inside 'upright', so it checked for thick eights. Matching whole words fixed it; the retrieval itself was never wrong.

WHY CHOOSE THIS?

Search by description, with the same encoders and no extra training.

WHY NOT THIS? (TRADEOFFS)

Only as good as the captions: an attribute never put in a caption (say, a closed four) cannot be searched for.

6. A combination never seen in training

DEFINITION — start here, no prior knowledge assumed

Can the pair handle a combination it never saw? A second pair of encoders is trained exactly as in section 3, except that a thick seven's caption never says 'thick'. It still says 'seven', and other digits are still called thick, so 'thick' and 'seven' are each known, but never together. Section 3's pair, which saw the combination, is the control.

REAL-WORLD SCENARIO

A child who has seen red apples and green cars, asked to point at a red car: if 'red' and 'car' were learned separately, the request still makes sense.

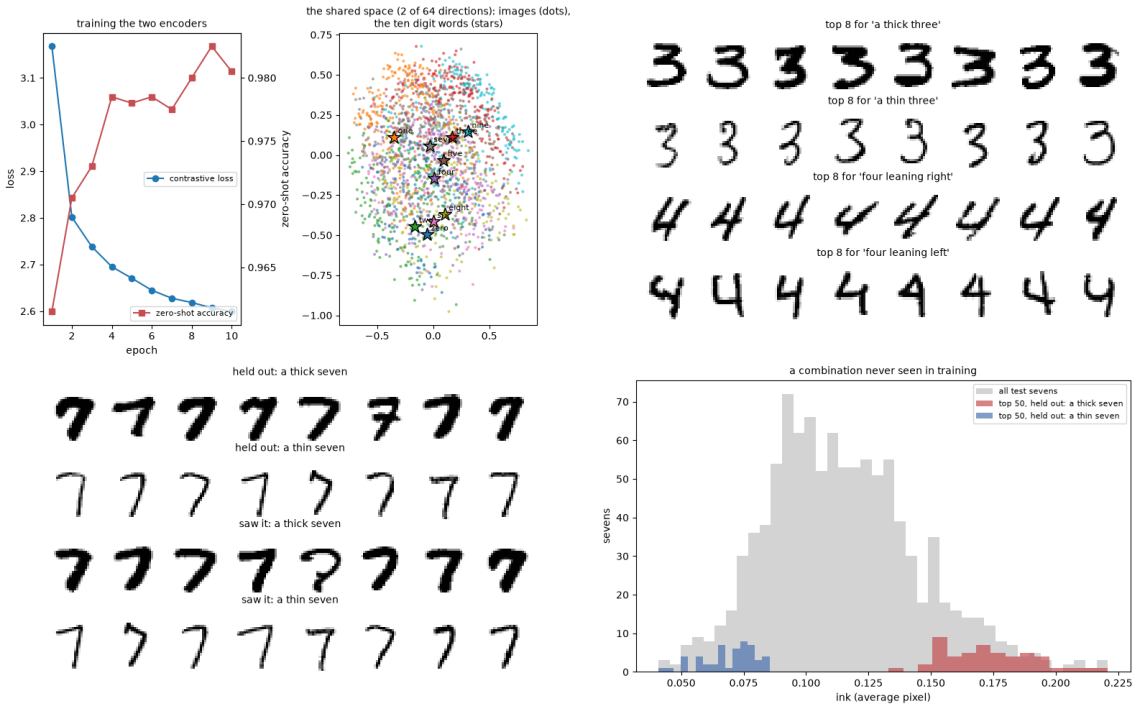
ACTUAL OUTPUT

```
a second pair of encoders, trained exactly as in section 3 except that a thick seven's caption never says 'thick'
(it still says 'seven', and other digits are still called thick). Section 3's encoders, which saw the combination,
are the control. Queries rank the 1,028 test sevens; thick share 0.33 is chance
(training sevens measured thick, never called thick: 2088; zero-shot accuracy of the held-out encoders 0.9785)
      encoders           query  thick share, top 50  mean ink, top 50
        saw it    a thick seven                 1.00            0.1620
        saw it     a thin seven                 1.00            0.0730
        saw it    a thick three                 1.00            0.2181
      held out    a thick seven                 1.00            0.1728
      held out     a thin seven                 1.00            0.0692
      held out    a thick three                 1.00            0.2164
all test sevens: mean ink 0.1149
Actual output from this lesson's run.
```

What it means

The held-out pair found thick sevens as well as the control did: all 50 of the top 50 were measured thick (1.00), with mean ink 0.1728, against 0.1149 for all test sevens; the control's top 50 had 0.1620. Keeping the word 'thick' out of the captions of all 2088 thick training sevens changed nothing measurable.

That is less surprising than it looks. With a bag of words, 'a thick seven' is the average of the word vectors, so the text side combines words by construction; the image side only needs thick sevens to lie in the same direction as thick digits of other kinds, which it learned from them. The histogram in the figure shows the two queries picking out the two ends of the sevens' ink distribution. A likely reason this works so cleanly, not tested here: thickness is close to a single number, the ink, that is easy to read from any digit.



Top: loss and zero-shot accuracy by epoch; the shared space flattened to its two main directions (dots: 2,000 test digits by class; stars: the ten digit words); the top 8 for four queries. Bottom: the top 8 sevens for 'thick' and 'thin', from the held-out and the control encoders; the ink of the top 50 against all test sevens.

WHY CHOOSE THIS?

Words learned separately combine into requests never seen together: the property text-to-image models rely on.

WHY NOT THIS? (TRADEOFFS)

Tested on one easy combination; harder ones (a property that changes a digit's shape) were not tried.

Interview preparation — technical questions

Q: Write CLIP's loss.

A: Unit vectors u, v; $S = u\,v^{T} / \tau$; cross-entropy of each row's softmax and each column's softmax against the diagonal, averaged. Here the code matched a hand computation to 0.996814.

Q: How do you classify with CLIP and no classifier?

A: Embed one prompt per class, embed the image, and take the most similar prompt. Here 0.9805 on 2,000 test digits.

Q: What does the temperature do?

A: Divides the similarities before the softmax: a small τ makes the softmax sharp, so the right pair must clearly beat the rest. 0.1 here, the only value tried.

Q: Why must the gradient go through the normalisation?

A: The loss sees only directions; the step $dz = (du - u(u \cdot du))/|z|$ removes the part of the gradient that would only change the length.

Q: How would you check an embedding captures an attribute?

A: Retrieve by that attribute and measure it: 'a thick three' found threes with mean ink 0.2181 against 0.1433 for all threes.

Interview preparation — theoretical questions

Q: Why is contrastive training label-efficient?

A: Every other pair in the batch is a free negative example; only 'these go together' is needed.

Q: Why can't the loss here reach zero?

A: Many captions in a batch are identical ('seven a'), so the right caption cannot be told from several wrong ones.

Q: What can't a bag-of-words text encoder do?

A: See word order or negation: reordered and 'not' prompts gave identical results here (cosine 1.000000; 'not a {}' 0.9805).

Q: Why does prompt wording matter more for real CLIP?

A: Its Transformer text encoder reads order and context, and web captions rarely look like 'a seven', so the wording that matches its training captions works best.

Q: What does compositional generalisation mean here?

A: 'Thick seven' retrieved thick sevens though no training caption said it: the two words were learned separately.

Key takeaway and what's next

KEY TAKEAWAY

A CLIP-style pair learns one space for images and words from nothing but matched pairs. Trained in 45 seconds on programmatic captions, it named test digits from words alone almost as well as a trained classifier, found digits by thickness and lean, and handled a combination it never saw. The bag-of-words text encoder sets the limits: word order, unknown words and negation change nothing. Day 144 uses this text encoder to steer a diffusion model.

- Zero-shot accuracy 0.9805 after 10 epochs (0.9615 after one); Day 138's classifier 0.9829.
- Nine prompt templates: 0.9745 to 0.9805; reordered words and unknown words: cosine 1.000000.
- Retrieval: right digit and right attribute in all top 50 for every query.
- Held-out 'thick seven': top-50 ink 0.1728, against 0.1149 for all sevens and 0.1620 for the control.

Suggested watch order, if starting from zero

- 1. OpenAI's CLIP Explained and Implementation | Contrastive Learning (Karndeep Singh) — CLIP's idea, with an implementation of it.
- 2. CLIP: Learning Transferable Visual Models From Natural Language Supervision (KnowledgeDB AI) — a walk through the CLIP paper itself.

Day 144 preview

- **Prompt engineering for generative models**: this text encoder steers Day 141's diffusion model, with classifier-free guidance, and prompts are varied to see what changes the images.